C++ 中为什么 new[] 和 delete[] 一定要配对使用？ 原创

发布于 2025-01-27 18:56:10 119 0 0 举报

在 C++ 中， `new[]` 和 `delete[]` 一定要配对使用，这是因为它们分别用于动态分配和释放数组的内存。如果使用不当，可能会导致内存泄漏、未定义行为或其他严重问题。

1. `new[]` 和 `delete[]` 的作用

- `new[]` ：用于动态分配一个数组，并返回指向该数组第一个元素的指针。
- `delete[]` ：用于释放由 `new[]` 分配的数组内存。

2. 为什么必须配对使用

2.1 内存管理

- **分配数组**： `new[]` 会为数组分配连续的内存块，并且会在内存块的某个位置存储数组的大小信息。这样， `delete[]` 在释放内存时可以知道要释放多少内存。
- **释放数组**： `delete[]` 会正确地释放整个数组的内存，包括数组的大小信息。如果使用 `delete` 而不是 `delete[]` ， 编译器 可能无法正确地释放数组的所有内存，从而导致内存泄漏。

2.2 未定义行为

- **使用 `delete` 释放 `new[]` 分配的内存**：这会导致未定义行为。编译器可能会尝试释放内存，但因为没有正确的数组大小信息，可能会导致部分内存未被释放，或者释放了不该释放的内存。
- **使用 `delete[]` 释放 `new` 分配的单个对象**：同样会导致未定义行为。 `delete[]` 会尝试释放一个数组的内存，而实际上只有一个对象的内存需要释放，这可能会导致程序崩溃或内存损坏。

3. 示例

正确使用 `new[]` 和 `delete[]`

代码语言：cpp 代码运行次数：0 运行 AI代码解释

```cpp
#include <iostream>

int main() {
    // 动态分配一个包含 5 个整数的数组
    int* array = new int[5];

    // 初始化数组

```

```cpp
    for (int i = 0; i < 5; ++i) {
        array[i] = i * 10;
    }

    // 输出数组内容
    for (int i = 0; i < 5; ++i) {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;

    // 释放数组内存
    delete[] array;

    return 0;
}
```

错误使用 `delete` **释放** `new[]` **分配的内存**

代码语言：cpp　代码运行次数：0　运行　AI代码解释

```cpp
#include <iostream>

int main() {
    // 动态分配一个包含 5 个整数的数组
    int* array = new int[5];

    // 初始化数组
    for (int i = 0; i < 5; ++i) {
        array[i] = i * 10;
    }

    // 输出数组内容
    for (int i = 0; i < 5; ++i) {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;

    // 错误地使用 delete 释放数组内存

```

```
    delete array;  // 这将导致未定义行为

    return 0;
}
```

总结

- `new[]` 和 `delete[]` **必须配对使用**：确保正确地分配和释放数组内存，避免内存泄漏和未定义行为。
- `new` 和 `delete` **用于单个对象**：确保正确地分配和释放单个对象的内存。

原创声明：本文系作者授权腾讯云开发者社区发表，未经许可，不得转载。
如有侵权，请联系 cloudcommunity@tencent.com 删除。

c++